

ENCM 511: Embedded System Interfacing
Assignment 2

Group 6

Chloe Fulbrook - 30210975

Ethan Lee - 30213225

Devon Calvin - 30204532

Submitted on October 10th, 2025

General Code Explanation:

For ENCM 511 Lab 2, our code is organized into two source files and a header. This allows us to create our functions separately from the main to assist with readability.

init.c

This file contains the function declarations for our IO initialization function, timer initialization function, and the delay function.

In `IOinit()`, we set all of our pins for the LEDs as outputs, and initialize the LEDs to off. We also set the global IOC interrupt controllers for use with the pushbuttons. The function then configures our buttons as inputs with pull-up resistors (since they are wired to ground) and sets the IOC controls for each pin. For PB0 and PB1, we utilize both positive and negative edge interrupts, allowing us to easily check if a button is being held. The idea here is that if we get a falling edge interrupt, the button has been pressed, and has not been released until we get a rising edge interrupt. For PB2, we are only using the negative edge interrupt because this button does not need to be held. It also helps us with our approach to debouncing PB2.

In `timerInit()`, we initialize timers 1, 2, and 3 for use in the lab. We are using T2 and T3 as independent 16-bit timers, not a single 32-bit timer, so this must be set before writing to the T2/T3 registers. For all three timers, we use a 1:256 prescaler, the internal clock as the clock source, and set a priority. We also need all the timers to continue running in idle mode to help us break the CPU's default idle states.

In `delay_ms()`, we write our function to create ms-timed delays in the program. It is important to note that the PR1 register of timer one is defined as a `uint16_t`, which limits the maximum possible delay for the function. The `uint16_t` has a maximum value $2^{16} - 1 = 65535$, which limits the tick count. With our 1:256 prescaler, this limits the millisecond count value to roughly $65535 * 0.064 = 4194.24$, meaning that the function can only be used for values in this range. This is a reasonable maximum for this application, and I see no need for a higher one. Within this function, we start by saving the current IEC register and then clearing it. This is an attempt to prevent other interrupts from breaking us out of the delay function; however, the timer one interrupt is enabled, which can break the idle. The next check is the overflow check, as described previously. Here, if a value above ~4190 is provided, the timer is set to count to its maximum instead. The timer control bits are set as required (see the code comments for more information), and the CPU is set to idle when the timer is enabled. Because of our IEC clearing, the only interrupt that can break this is the T1 interrupt itself, so we eventually fire the T1 ISR that clears the flag and turns off the timer again. Upon returning to the delay function, we restore the saved IEC and disable the T1 interrupt again.

init.h

The `init.h` file is the header file that contains the function prototypes for the IO and timer initializations, as well as the delay. It also holds the `#define` macros for the LED pins.

main.c

The main file holds the majority of our code. It begins with the `#pragma` for setup and configuration. We then include the header described above, along with the `xc.h` header. Some `#define` macros are set for our pushbuttons, a debounce constant, and the 0.25 and 0.5s timer periods. Volatile flags are used to track button events and the debounce state of PB2. We create an array of blink rates for LED1, indexed by the `PB2_times_index` variable, which is incremented by PB2 events.

From there, we enter into the primary function, beginning with the initialization of our IOs and timers, as described above. We create enumerated state names for our Moore FSM, and default the state to the off state. Event flags are then cleared, and we begin our `while(1)` superloop that immediately starts a switch statement that controls the FSM. Upon entering any case, we set the CPU to idle and introduce a short delay upon exiting idle. This helps ensure that the button states have stabilized before reading their values, which in turn assists us in making sure the state transitions are cleaner. Within each case, we handle the state transitions based on the flags set by the IOC ISR. If a flag is set, we set up that state and transition to it. The finer details of these transitions are commented on in more depth in the main file. The one outlier to this is in the PB1 state, where we check the PB2 event flag to see if the count duration has changed while PB1 is still being held—main ends as the case statement ends.

The file continues to handle our interrupt service routines. The IOC ISR handles our button interrupts. The PIC chip has only one IOC ISR, so we need to determine which pin triggered it. The ISR is entered anytime an interrupt occurs on the button pins, as set by our IO initialization. We can check the status registers for PORTA and PORTB to determine which port was interrupted. We set the flags based on this, and the edge detection is set in the `IOinit`. We also clear the port status flags each time we enter the ISR, as well as the global IOC interrupt flag. The PB0 and PB1 interrupts are straightforward, setting the flag when the button is pressed and clearing it when the button is released. For PB2, we verify that the button is pressed and that debouncing is not currently occurring. If this is true, we set the debouncing flag and start timer 2 for 50ms to allow the PB2 button input to stabilize and avoid multiple triggers of the ISR. As the timer is set, we enter the T2 ISR, which immediately disables the timer, clears the interrupt, and sets the flag. It then ensures that the button is still pressed and flags an event. It increments the `PB2_time_index`, handles array index overflow, and resets the debounce progress flag to indicate that debouncing is not currently in progress. This helps store the LED1 blink timing, updating it whenever the button is pressed and saving the value.

The T1 ISR is straightforward; it simply clears the flag and disables the timer for use in the delay function (described earlier).

The T3 ISR handles which LED should toggle upon reaching the T3 count. It first clears the interrupt flag, then proceeds to check which buttons are having events (pressed), and toggles the corresponding LEDs.

Special Function(s):

Delay function

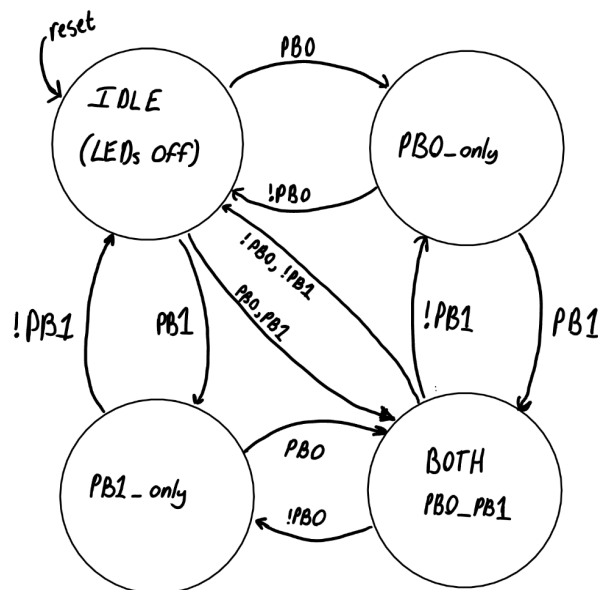
Parameters: uint16_t ms

Arguments: Length of delay in ms ($0 < \text{ms} < 65535$)

Timer counting calculation = *Desired time* * $\frac{\text{Clock Frequency}}{\text{Clock Scaling}}$

The implementation and execution of the delay function is answered in the questions section.

Our Moore-type FSM:



Questions:

(i) In the original provided program, why does the LED appear to “flash” every time you “click” a button (quickly)?

When a tactile button, such as the one used in ENCM511, or any mechanical switch, is pressed quickly, it bounces between its mechanical contacts, thereby changing its logic states. As the button switches between the threshold voltages required for logic one and logic 0, the microcontroller recognizes many rapid switches rather than a single press in a single state. Because the microcontroller interprets the fast transition, it signals the LED to “flash” every time you click the button quickly.

(ii) If you press a button and hold it there, does the IOC interrupt service routine keep executing repeatedly? If so, why? If not, why not?

The ISR will not continually execute if the button is continuously held down. This is because IOC is only triggered when the pin’s voltage level changes, specifically when it is pressed or released, depending on the IOC's configuration. It is triggered by edge detection in our code. Additionally, we clear the status flags and global IOC flags during the ISR to prevent continuous triggering of the ISR.

(iii) Explain your implementation of `void delay_ms(uint16_t ms)`. How do you guarantee that the delay works across the range of possible input values? What combination of clock/prescaler/PRx values do you use? What is the maximum delay you can use for the FOSC and timer prescaler setting?

The delay function is implemented by using Timer1 and an `idle()` function. Within the function, PR1 is calculated and set using the formula $PR1 = (ms / 0.064)$. The IEC is saved and set to 0, with only the Timer1 interrupt enabled, meaning that the only interrupt available to break us out of `idle` is the one set created by Timer1 itself. Timer1 start count is set to 0, and the saved IEC register is restored at the end of the delay function. Timer1 is started within the delay function, where it then hits an `idle()` and waits until the timer counter reaches PR1, which causes the Timer1 ISR to execute. Within the ISR, Timer1 is turned off, and flags are reset. Then it heads back to the delay function, where it re-enables interrupts (restores the IEC). The delay ranges from 0 to 4194ms and is maintained within this range by incorporating if statements that check if it exceeds the maximum possible value, setting it to the longest possible value if true. The PIC24 uses an 8MHz clock with a 1:256 prescaler, and PR1 can range between 0 and 65535. The maximum delay possible with our configuration is 419 ms.

(iv) How many timers do you use? What do you use each timer for?

We use all three available timers. Timer1 is used to set the duration of a delay within the delay function. Timer2 is used as a slight delay to give PB2 enough time to hold a stable value for the variable blink rate. This allows us to debounce Timer2 to ensure that only one press is registered

for each click. Timer3 is used to control the blink rates of the LEDs and which LED should toggle.

(v) Explain your implementation of the IOC interrupt service routine, and your choice of IOC settings. How do you distinguish between a button press and a release? How about the different buttons?

The IOC ISR checks the input of buttons being pressed and sets global event flag variables depending on the state of the button ports. In the IOC ISR, an event on any of the buttons is read, and a flag is set. The PB0 and PB1 have fundamental implementations, where a flag is set if a negative edge is detected, and cleared if the edge is positive. The PB2 press initiates a debouncing Timer2, which controls the list index for the PR3 values that determine the blink rate of LED1, provided PB1 is held.

For IOC settings, PB1(RB4), PB0(RA4) trigger interrupts on both positive and negative clock edges. PB2(RB9) triggers on the negative clock edge only. Within the ISR, if statements check the states of each button and set the corresponding event flag to high when the button is low and to low when it is high. It distinguishes different buttons by checking the IOCSTATbits.IOCPUF, which helps determine if the interrupt occurred on the A or B ports. The state of the individual buttons associated with each port is then checked within the if statement. For our implementation, we check the state of the pin after an edge is detected. After a negative edge, we see that the pin is low, and vice versa, setting or clearing flags accordingly. For buttons 0 and 1, as long as the event flag is set, the button is actively being pressed. Button 2 only triggers on the negative, setting the flag. And within the status registers of the IOC, we can determine which button triggered the interrupt based also on the level of the pin.